

TWO WHEEL SELF BALANCE ROBOT

Table of Content

Introduction	3
Objectives:	3
Background/Theory	4
Inverted Pendulum Theory.....	4
PID Control	4
Methodology/System Design	6
Hardware Selection	6
Mechanical Design	7
Electronics / Wiring considerations.....	8
Software design	9
Implementation.....	11
Planning & Schematic Diagram	11
Procurement.....	11
Circuit Testing Before Soldering.....	12
Soldering Process	14
Verification	14
Frame making.....	15
Assembly Frame and circuit	16
Programming.....	16
Code explanation.....	18
Implementation Attempt with Complementary Filter and PID Library (PID1.4)	18
Implementation Attempt with MPU6050 DMP and Open-Source PID Framework	19
Progress Journal – Self Balancing Robot	21
Conclusion	24
Reference:	25
Appendix	26
Schematic Diagram.....	26
Perfboard layout (30x24).....	26
3D Modeling Solidworks.....	27
PID_1.4	27
PID_2	34

Introduction

Balancing a two-wheel robot is a classic challenge in robotics that demonstrates the integration of mechanics, electronics, and control theory. The system behaves like an inverted pendulum — an inherently unstable configuration that requires continuous correction to remain upright.

At the heart of the self-balancing robot is an Inertial Measurement Unit (IMU), which provides real-time data on the robot's tilt angle and angular velocity. These measurements are used as inputs to a closed-loop feedback controller. In this project, Proportional–Integral–Derivative (PID) controller, a widely used control method in automation, is employed to calculate corrective motor commands that stabilize the robot.

The ESP32 microcontroller serves as the main processing unit. Its fast clock speed, built-in Wi-Fi/Bluetooth, and support for multiple communication protocols (such as I²C and PWM) make it suitable for handling both real-time control loops and future feature expansions, such as wireless monitoring or remote control. The motors are driven through an H-bridge motor driver, powered by a rechargeable battery pack.

This documentation records the process of building the robot step by step, covering hardware selection, sensor integration, PID control implementation, mechanical design, wiring considerations and also the challenges faced along the way.

Objectives:

To demonstrate the use of a Proportional–Integral–Derivative (PID) controller to stabilize a naturally unstable system.

Another objective is to learn how to integrate an Inertial Measurement Unit (IMU) with the ESP32 microcontroller. This includes not only establishing communication over the I²C protocol but also processing raw accelerometer and gyroscope data to obtain accurate tilt angles.

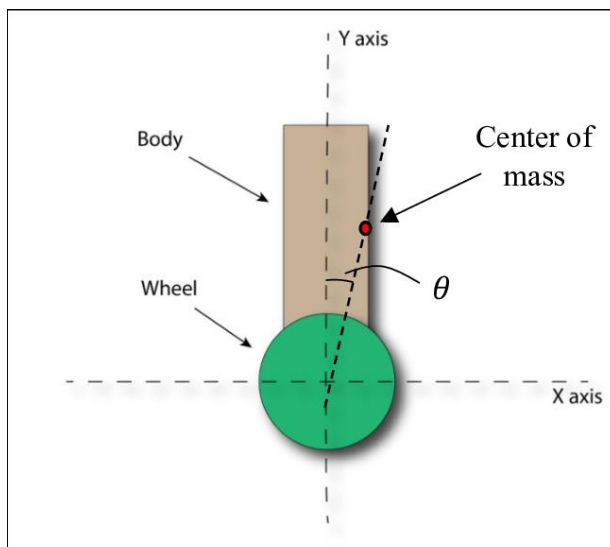
From a mechanical and electrical perspective, the project aims to design and assemble a compact mechatronic system. This includes constructing the frame, integrating motors and wheels, and ensuring that the motor driver, battery, and ESP32 are properly connected and safely powered.

Background/Theory

Inverted Pendulum Theory

The self-balancing robot can be modeled as an inverted pendulum mounted on a moving base. Unlike a normal pendulum that naturally swings back to its lowest position, an inverted pendulum is inherently unstable because its equilibrium point lies at the top. If the pendulum tilts even slightly, gravity creates a torque that accelerates it further away from the upright position. In the case of the robot, its body acts as the pendulum while the wheel axis serves as the pivot. To keep the pendulum upright, the wheels must continuously move forward or backward to generate corrective forces. This dynamic resembles how a person balances on a bicycle or when standing still, making the inverted pendulum a classic benchmark problem in control theory.

Interestingly, the center of gravity (CoG) plays an important role in the robot's stability. In an inverted pendulum system, a slightly higher CoG can actually make balancing easier. This is because a taller robot has a slower angular acceleration when tilted—the fall is less abrupt, giving the control system more time to detect the tilt and respond. In other words, increasing the distance between the pivot point (wheel axis) and the CoG increases the system's moment of inertia, which reduces how quickly the robot falls. This makes the balancing problem more manageable, though it still requires precise control.



PID Control

$$\text{Output} = K_P e(t) + K_I \int e(t) dt + K_D \frac{d}{dt} e(t)$$

Where : e = Setpoint - Input

To stabilize the inverted pendulum, a feedback controller is required. One of the most commonly used methods is the **Proportional–Integral–Derivative (PID) controller**.

- The **Proportional (P) term** provides an immediate correction that is proportional to the size of the error.
- The **Integral (I) term** accumulates small errors over time and corrects steady-state offsets
- The **Derivative (D) term** reacts to the rate of change of the error.

When properly tuned, PID combines these three effects to create a balance between responsiveness, accuracy, and stability.

Why PID is widely used in Control System

PID controllers are one of the most widely used control methods in engineering and industry because of their **simplicity, effectiveness, and versatility**. The mathematics behind PID are relatively straightforward, yet they can handle a wide variety of control problems without requiring a complex system model.

PID is commonly applied in systems that require **continuous regulation** toward a desired setpoint. Examples include:

- **Temperature control** (ovens, air conditioners, furnaces, 3D printers).
- **Speed control** (motors, fans, conveyor belts).
- **Position control** (robot arms, CNC machines, drones).
- **Process control** in chemical plants, water treatment, or pressure regulation.
- **Balancing systems**, like inverted pendulums, Segways, and self-balancing robots.

One of the key reasons PID is so popular is because many real-world systems—whether thermal, mechanical, or electrical—have dynamics that can be sufficiently stabilized with just proportional, integral, and derivative actions. Unlike advanced control methods that require deep mathematical models (like state-space or adaptive control), PID can often be tuned experimentally, making it very practical for engineers and hobbyists.

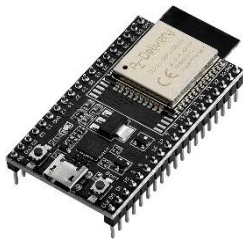
Another reason is that PID works well in **repetitive or steady-state tasks**, where the system needs to repeatedly correct itself to maintain stability (e.g., holding temperature, maintaining speed, or keeping balance). It provides a good trade-off between responsiveness, accuracy, and stability, which explains why more than 90% of industrial controllers in use today are still PID-based.

Methodology/System Design

Hardware Selection

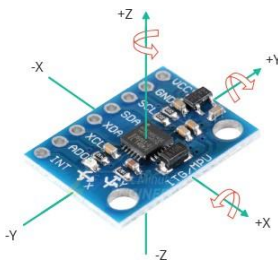
Microcontroller

ESP32 Doit Dev kit V1 – for its fast processing speed and integrated Bluetooth/Wi-Fi, making it capable of handling real-time control loops while allowing room for future feature expansion



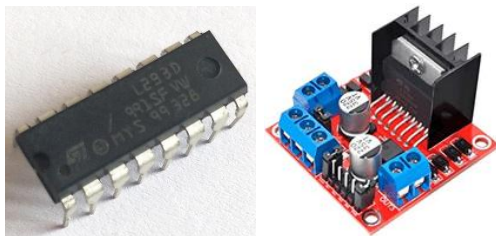
Inertial Measurement Unit (IMU)

MPU6050 - to measure tilt angle through accelerometer and gyroscope data. It is a common choice in self-balancing robot projects, with abundant tutorials and references available



Motor driver

L293D - to control the DC motors. While the L298N is more commonly used due to its higher current capacity, built-in heat sink, and voltage regulator, the L293D was selected here as it was readily available from a previous project.



Motor

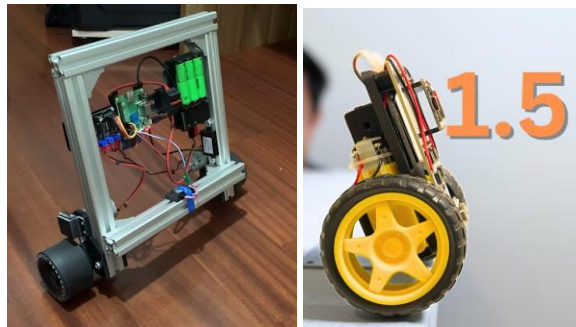
Geared DC motors - to increase torque output, as raw brushed DC motors provide insufficient torque for balancing tasks. For improved performance in future iterations, stepper motors with encoders could be used to achieve more precise control and closed-loop feedback.

Mechanical Design

During the research phase, I identified two common types of self-balancing robots:

1. Non-upright

- The center of gravity is not strictly vertical to the wheel axis, which makes it visually appealing



2. Conventional upright type:

- Built with two wheels aligned along the same axis.
- The chassis is usually rectangular with 2–3 stacked layers.
- The system balances vertically upright, making it a closer representation of the inverted pendulum model.



For this project, I chose the **conventional upright design** because calibration and PID tuning are simpler and more consistent.

The frame is constructed using **double-wall cardboard**, selected for its ease of prototyping and additional structural strength compared to single-wall cardboard. The pieces are assembled with super glue to ensure rigidity and stability.

The component layout across the layers is as follows:

- **First layer:** ESP32 microcontroller and L293D motor driver.

- **Second layer:** Battery pack (the heaviest component), positioned here to raise the system's center of gravity. A higher center of gravity makes the robot's fall slower, giving the control system more time to react.
- **Third layer:** MPU6050 IMU sensor, placed near the center to ensure accurate tilt measurement with minimal external vibration effects.



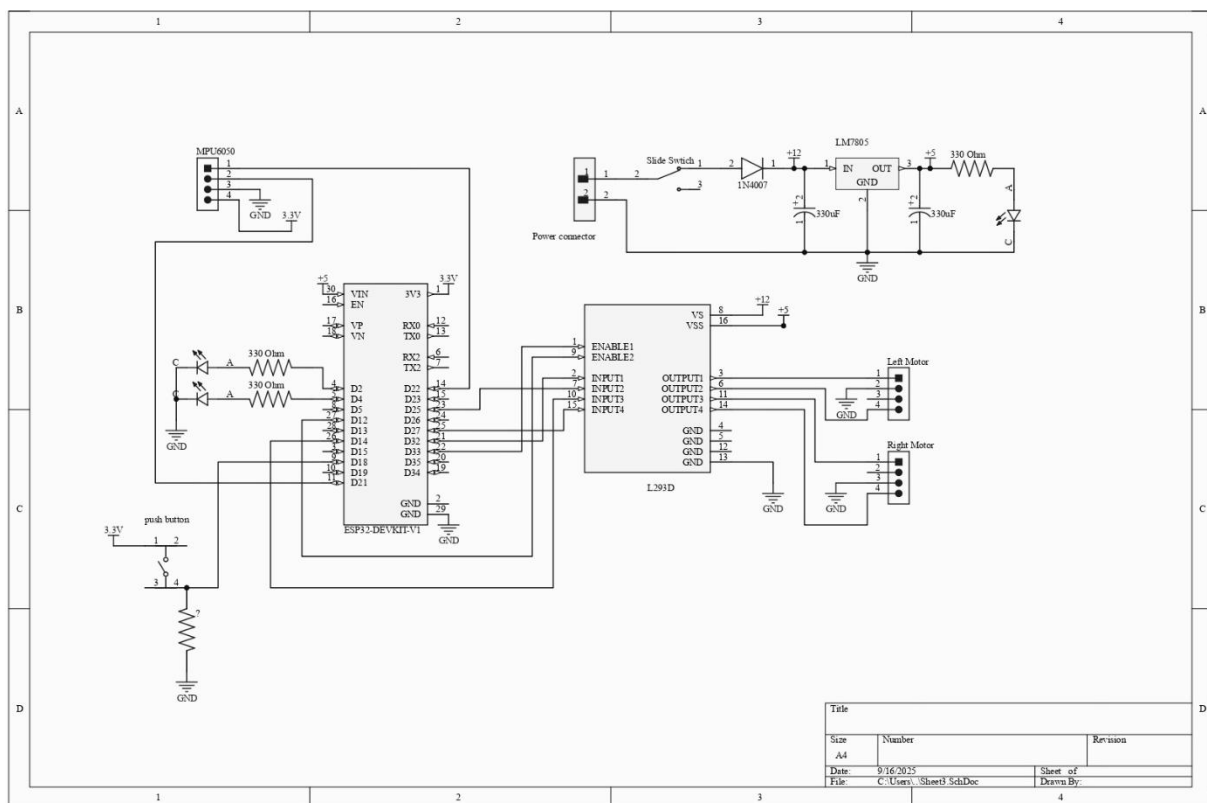
Electronics / Wiring considerations

- Circuit diagram
- Power supply design (voltage, current needs, battery charging).

The robot is powered by two 3.7 V Li-ion cells connected in series to provide approximately 7.4–7.7 V. A slide switch is placed on the battery positive lead to turn the entire system on and off, followed by a 1N4007 diode for basic reverse-polarity and backflow protection. The motor supply (V_{motor}) uses the raw series battery voltage so the two DC gear motors receive the full 7.7 V for sufficient torque.

A linear 5 V regulator (LM7805) steps the battery voltage down to a stable 5 V rail that feeds the ESP32 DevKit via its VIN pin and also supplies the logic VCC for the L293D driver. The ESP32's GPIOs (3.3 V logic) drive the L293D inputs directly; 3.3 V is recognised as a valid HIGH by the L293D logic inputs. The L293D H-bridge handles bi-directional motor control (forward/reverse) under commands from the ESP32. Capacitors are placed close to the regulator input and output to stabilise the supply. Ensure a common ground between battery, ESP32, and motor driver.

Schematic Diagram



Software design

1. Control Algorithm

The self-balancing robot uses a closed-loop control system based on the Proportional–Integral–Derivative (PID) controller.

The input is the tilt angle of the robot, measured by the MPU6050 sensor. The setpoint is the upright angle (around 180° in the DMP implementation). PID controller continuously computes an output value based on the difference between the setpoint and the actual tilt. The output determines the PWM duty cycle applied to the motors via the motor driver (L293D), which moves the robot forward or backward to restore balance.

2. Flowchart of Control Loop

The flowchart should follow this sequence:

1. Start → Initialize peripherals (MPU6050, PID, Motors) → Read sensor values
2. Calculate tilt angle (sensor fusion) → Compute error (setpoint – angle)
3. Apply PID algorithm → Determine motor speed and direction
4. Send PWM signals to motor driver

5. Repeat loop continuously
-

3. Sensor Fusion

Two approaches were explored:

a) **Complementary filter (custom implementation, PID_1.4):**

Accelerometer and gyroscope data are read directly from the MPU6050.
The filter combines accelerometer (long-term stability) and gyroscope (short-term accuracy) data to estimate the robot's tilt angle (pitch).
This angle is compared against the desired setpoint (upright position).
PID controller computes the corrective output, which is then mapped to motor speed and direction.

b) **DMP (Digital Motion Processor, PID_2):**

The MPU6050's internal DMP is enabled to offload sensor fusion calculations (quaternion → yaw, pitch, roll).
Pitch angle is directly obtained from the DMP output, reducing noise and simplifying implementation.
Tuned PID controller then drives the motors to maintain balance.
This method provided more stable long-term balancing compared to the complementary filter approach.

4. PID Tuning

- The PID gains (K_p , K_i , K_d) are tuned experimentally.

Implementation

Planning & Schematic Diagram

Before procurement, a detailed schematic was drawn to plan the complete circuit and identify all required parts. The schematic ensured that pin assignments, power rails, and signal connections were clear. Please refer the Schematic Diagram from electronic Design on previous section of Methodology.

Procurement

I bought most of them in Shopee and local electronics store

No.	Component	Quantity
1	DOIT ESP32 DEVKIT V1 Board	1
2	GY-521 MPU6050	1
3	L293D Motor Driver	1
4	LM7805 Linear Voltage Regulator	1
5	Slide Switch	1
6	2-pin Header	1
7	1-pin Header	1
8	330 μ F Capacitor	2
9	330 Ω Resistor	3
10	4.7 k Ω Resistor	1
11	1N4007 diode	1
12	LED	3
13	Push Button	1
14	IC Socket (8x4)	1
15	15-pin Female Header	2
16	4-pin Female Header	2
17	3.7V Rechargeable Li-ion Battery	2–4
18	18650 Li-ion USB Charger	1

19	18650 Double Cell Battery Holder	1
20	3V–6V Dual Axis TT Gear Motor	2
21	TT Motor Wheels	2
22	Ice Cream Sticks	-
23	Cardboard (double wall preferred)	-
24	Single-sided PCB Donut Board (7×9 cm)	1
25	Jumper Wires (female)	5
26	Jumper Wires (male)	4
27	Solder	-

Tools needed

1. Masking tape
2. Soldering iron
3. Super glue
4. Desoldering pump, flux
5. Cutter, Scissors

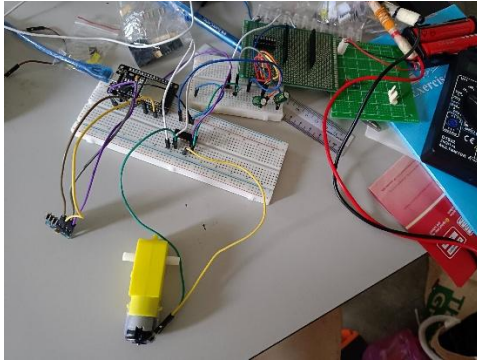
A key lesson during procurement was the importance of verifying component compatibility and stock quality. For example, the first MPU6050 module purchased was defective and wasted significant debugging time until it was replaced with a functioning unit. Similarly, the wrong type of perfboard (double-sided and too small at 7×5 cm) was initially bought, which was not suitable for line-to-line solder connections. Later, a 7×9 cm single-sided donut board was selected, which made soldering and circuit tracing much easier due to its flux-coated surface.

It is also recommended to buy extra components for critical modules such as sensors, wires, and connectors, as some local shops may carry old stock or incorrect variations. In addition, it is necessary to carefully check the part specifications (e.g., resistor values, IC socket size, header pin spacing) to avoid mismatches

Circuit Testing Before Soldering

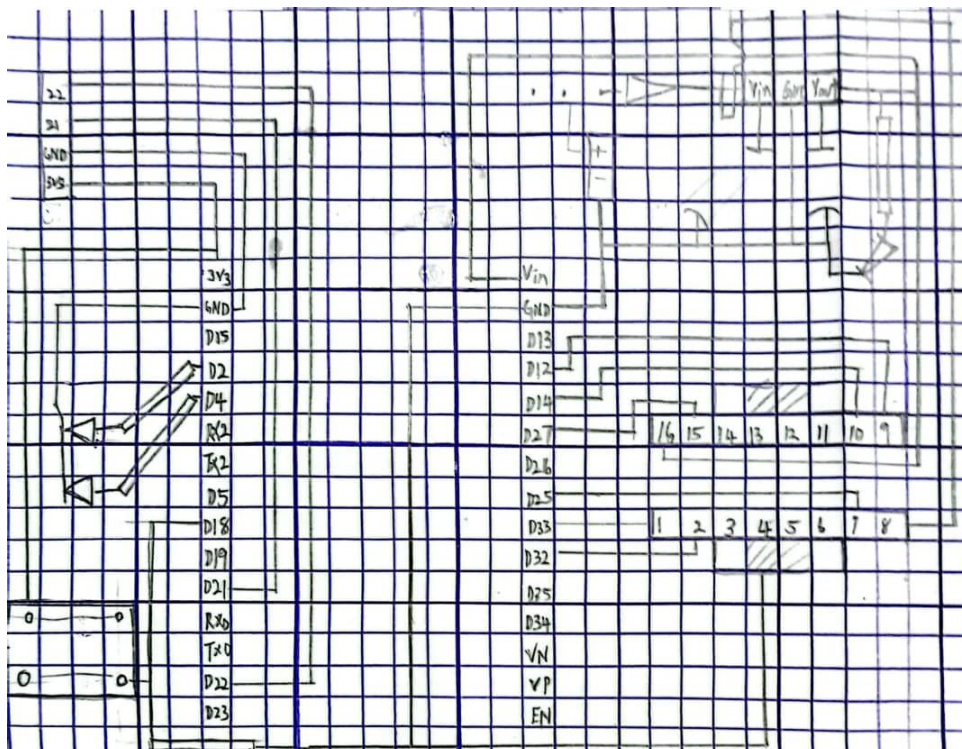
After completing the procurement, the next step is to verify the components individually and as a system before committing them onto the perfboard. Using the schematic as a reference, the circuit was assembled temporarily on a breadboard to allow for easier debugging and modifications.

The power management stage was tested first by supplying the batteries through the LM7805 linear regulator and confirming a stable 5V output. The MPU6050 IMU module was then connected, and readings of the accelerometer and gyroscope were checked in serial monitor.



Next, the motor driver (L293D) was tested by driving each TT geared motor separately. Special care was taken to select two motors that demonstrated nearly identical speeds when supplied with the same voltage. This matching process was important to reduce imbalance in the self-balancing robot during operation. The PWM control from the ESP32 was also verified to ensure smooth speed variation.

After confirming that all components worked as expected, the physical placement of parts on the single-sided perfboard was planned. The perfboard layout was first sketched on graph paper (square grid paper) to replicate the hole spacing and allow proper visualization of component placement.



(30x24)

Soldering Process

The components were soldered onto the single-sided perfboard according to the planned perfboard layout. The soldering iron I used was a Yihua 928D-I, which supports adjustable temperature control.

For solder material, a lead-based solder was chosen due to its lower melting point. During soldering, a damp sponge or brass wool was kept nearby to regularly clean the soldering tip. Proper tip maintenance ensures efficient heat transfer and prolongs the lifespan of the iron. It is also recommended to tin the tip before touching the joint, as the thin solder layer improves heat transfer.

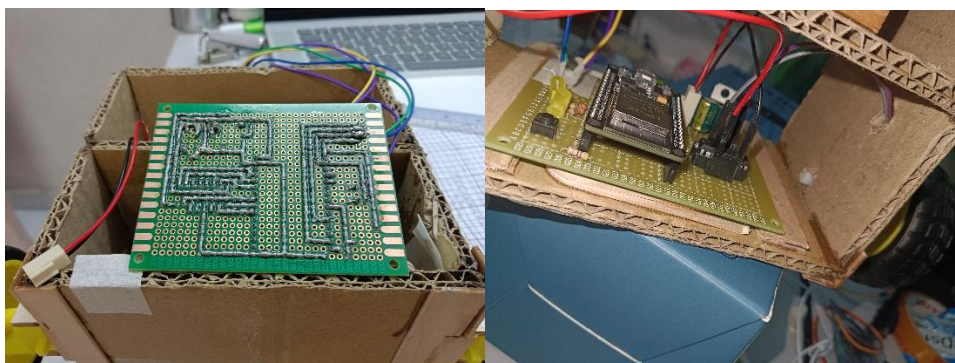
The temperature setting plays a crucial role in achieving strong and clean joints. While most components solder well at moderate temperatures, components like the LM7805 voltage regulator with its heat sink require a slightly higher temperature to allow solder flow. After completing a joint, it is good practice to leave a thin layer of solder on the tip to protect it from oxidation.

Finally, once all components were soldered, the soldering tip was coated with solder before storage, ensuring long-term protection against oxidation.

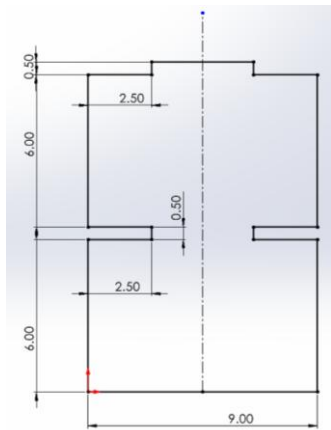
Verification

A test was carried out to ensure the safety and reliability of the circuit before applying power. Using a digital multimeter, all critical checks were performed:

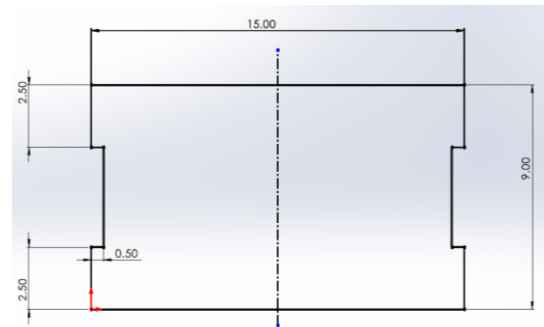
- Continuity testing was used to verify that all pin-to-pin connections matched the schematic design.
- Short-circuit testing was done across the power rails (VCC to GND) and between sensitive pins to confirm that no unintended solder bridges or shorts were present.



Frame making



Board A x2



Board B x3

The frame of the robot was constructed primarily from double-wall cardboard for its light weight and reasonable strength. 2 cardboard pieces were cut according to dimension Board A to serve as the left and right side supports. In addition, 3 cardboard pieces were cut according to Dimension B and glued together using super glue to form a central structural block, providing rigidity to the frame.

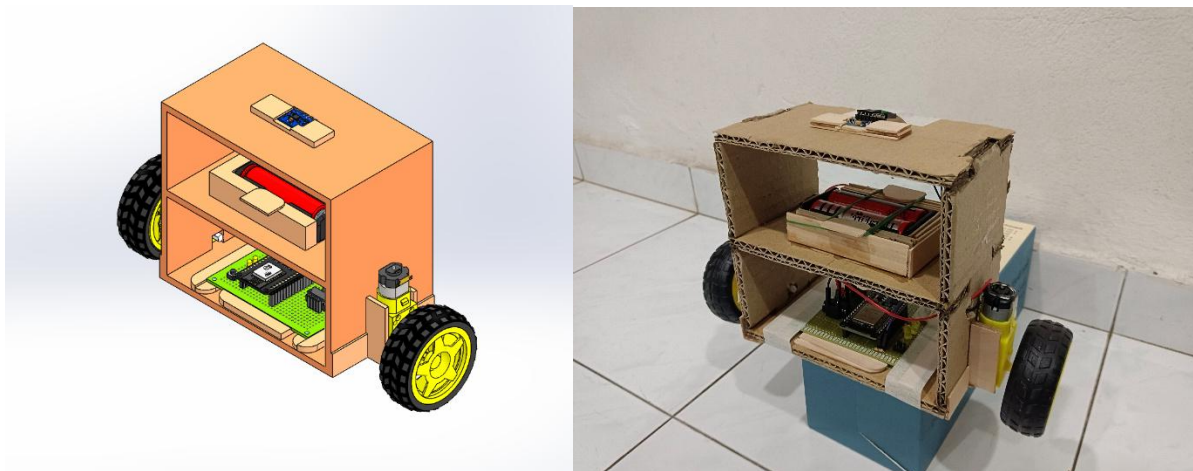
To mount the electronic components, ice cream sticks were used as holders for the circuit board, battery holder, and MPU6050 module. These components are placed near the center line of the frame to maintain balance. For temporary mounting and easier disassembly during debugging, rubber bands and masking tape were used to attach components securely while still allowing adjustments.

The motors were installed symmetrically on the left and right sides of the frame, positioned at the bottom center to ensure stable weight distribution. A hole was cut opposite each motor to allow clearance for the idle shaft of the TT gear motors.



Assembly Frame and circuit

Once the frame was completed, the next step was to assemble and integrate the electronics. The circuit board was mounted securely onto the frame, and the yellow DC motors were soldered with wires before being connected to the motor driver (L293D) header pins. The IMU sensor (MPU6050) was placed at the top of the robot to provide stable orientation readings, and a battery holder was attached to supply power, fastened with a rubber band for extra stability. With the mechanical setup and wiring completed, the robot was ready to move on to the programming stage.



Programming

The implementation begins with downloading the required libraries and placing them inside the Arduino libraries folder. Next, include the libraries in the sketch and adjust the pin configurations if necessary. Once the basic setup is complete, proceed with PID tuning.

Before tuning, ensure the wheels rotate in the correct direction. If they move incorrectly, swap the motor wires. Observe the robot's behavior:

- If the robot reacts too slowly → increase K_p .
- If the robot overreacts or oscillates → decrease K_p .
- If the robot consistently falls over and cannot recover → increase K_i .
- If the system oscillates too sharply or overshoots → adjust K_d .

During the process, also verify the calibration. If the robot always falls toward one side, two possible causes are: (1) the floor surface is not level, or (2) the MPU6050 calibration is biased. In the latter case, recalibration is necessary.

PID Tuning in Self-Balancing Robots

Proportional (K_p): Determines how strongly the robot reacts to the error.. If too high, oscillations occur; if too low, response becomes sluggish.

Derivative (K_d): Reacts to the rate of change of the error. It provides damping by applying an opposite corrective force when the robot falls quickly

Integral (K_i): Accounts for the accumulation of error over time. It helps eliminate steady-state error and improves recovery, but too much K_i causes drift and instability.

Personal Tuning Approach

From my tuning experience, K_p and K_i play the significant role in achieving balance. I begin by gradually increasing K_p until the robot can respond to balance, then lowering it slightly to find the smallest stable value. Once K_p is optimized, I introduce K_i to improve recovery and prevent drift. Finally, I adjust K_d to provide damping, keeping it at the optimum level that avoids overshoot.

Code explanation

For the coding phase, I first attempted to develop the program on my own with the aid of ChatGPT. This initial version achieved a maximum balancing time of approximately 3 seconds. To improve performance, I then transitioned to an open-source implementation that had been shared online.

In the Code Explanation section, I will present and analyze both approaches, which are included in the appendix:

PID_1.4 - my self-developed code

PID_2 - the open-source reference code

Implementation Attempt with Complementary Filter and PID Library (PID1.4)

Approach

In the initial attempt, the balancing code was developed by combining:

- Adafruit MPU6050 library to obtain raw accelerometer and gyroscope readings.

- Complementary filter to fuse the two sensor inputs and estimate the pitch angle.

- PID_v1 library to calculate corrective motor commands based on the pitch angle error.

The motors were controlled directly with forward/reverse functions, while PWM duty cycles were scaled according to the PID output.

Observations

- The complementary filter provided a reasonable pitch estimate, but with some drift and noise.

- Initial trials with a manually written PID loop did not yield stable results, so the PID_v1 library was adopted for tuning.

- With this setup, the robot managed to balance for about 3 seconds before gradually losing stability and oscillating heavily.

- Adding a small delay (35 ms) in the main loop unexpectedly improved performance, as it slowed down the control loop and reduced rapid oscillations. However, this improvement was only temporary and did not solve the underlying instability.

Analysis

- The improvement with delay(35) indicates that the control loop was running at an inconsistent or overly fast frequency. The delay acted as a crude timing control, effectively slowing the update rate.

While this gave the impression of stability, the PID gains were not tuned for a consistent, high-frequency control loop (typically 100 Hz).

The lack of consistent timing and proper tuning limited performance, causing the system to fall after a short duration.

Lessons Learned

PID tuning must match the control frequency. Gains tuned with an artificial delay will not work once the delay is removed.

A fixed-rate control loop (e.g., 10 ms = 100 Hz) is more reliable than using arbitrary delays.

Transition to Open-Source Reference Code

After these trials, the development shifted to using an open-source self-balancing robot implementation that integrates the MPU6050's Digital Motion Processor (DMP). This approach improves sensor fusion accuracy, ensures consistent timing, and provides a more robust framework for PID tuning.

Implementation Attempt with MPU6050 DMP and Open-Source PID Framework

Approach

Key elements of this implementation:

- I2Cdev and MPU6050 libraries (by JRowberg) to initialize and configure the DMP.

- Interrupt-driven data handling using the MPU6050's INT pin for timely FIFO reads.

- FIFO buffer to store complete sensor data packets.

- PID_v1 library for control calculations based on the pitch angle (from DMP).

- LMotorController library to simplify motor direction and PWM handling.

The PID loop computes corrective outputs, which are then applied to the motors. The system continuously reads pitch values from the DMP and adjusts motor speeds to maintain balance.

Observations

The DMP provided much cleaner and stable angle readings compared to the complementary filter approach.

With interrupts and FIFO buffering, the control loop became more consistent and less dependent on arbitrary delays.

The robot showed noticeably better balancing performance than the first attempt, though still sensitive to tuning of PID constants.

Occasional issues such as FIFO overflow were observed, usually caused by delays in reading data quickly enough.

Analysis

By using the DMP, the robot avoided the drift and noise issues seen in the complementary filter method.

The interrupt-driven structure allowed the PID loop to run with more consistent timing, which is crucial for stable balancing.

PID tuning remained the most critical factor for performance. Too high Kp caused rapid oscillations, while incorrect Ki values led to drift or instability.

FIFO overflow indicates that the processing loop must keep pace with the DMP output rate. This can happen if the loop is slowed down by serial printing, heavy calculations, or blocking functions.

Lessons Learned

Offloading sensor fusion to the DMP significantly improves stability and frees processing time for control tasks.

Ensuring the PID loop runs at a consistent frequency is just as important as choosing the right PID gains.

Debugging tools such as serial plotting are useful but should be minimized in real-time runs to prevent FIFO overflows.

A structured library-based approach (I2Cdev, LMotorController, PID_v1) simplifies development and reduces errors compared to writing everything from scratch.

Overall Reflections

Step 1 (Complementary Filter): Built a working baseline, taught the importance of timing and filtering.

Step 2 (DMP + Libraries): Significantly improved stability by leveraging hardware sensor fusion and structured code libraries.

The project highlights how incremental improvements — from raw filtering to optimized libraries — can transform performance.

Future work should focus on systematic PID tuning and possibly implementing features like auto-calibration or adaptive gains.

Progress Journal – Self Balancing Robot

Week 1

Day 1 (19/6)

Purchased essential components and tools.

Found a local electronics supplier

Picked up additional tools from MR DIY.

Realized some parts from local store (IC socket, pin headers) were incorrect → note to always double-check parts before leaving the store.

Day 3

Reviewed theory: push–pull configuration, active high/low, pull-up/pull-down resistors.

Experimented with ESP32 and MPU6050.

Faced issues:

Wrong components from supplier.

Soldering tool maintenance problems.

Day 4

Studied MPU6050 tutorials.

Found the bought IMU not usable (WHOAMI showed 0x70 instead of 0x68).

Day 5

Tested Kalman and Complementary filters.

Day 6

Completed schematic and perfboard plan on paper.

Old soldering iron failed → purchased a new one.

Week 2–3

Day 10 (29/7/2025)

Tried soldering the complete board.

Issues: fast oxidizing soldering iron, unsuitable protoboard



Day 12

Motor testing with ESP32 PWM.

Learned ESP32 uses a newer PWM API.

Day 19

Wrote simple PID control.

Found motors need $\geq 75\%$ PWM to operate effectively.

Implemented proportional control logic.

Day 25 (13/8)

All components ready.

Confirmed motors require high PWM (75–100%) for reliable torque.

Week 6

Day 41 (29/8/2025)

Used chassis parts cut earlier

Assembled with super glue, fixed motor wiring.

MPU6050 wire broke due to the four inconsistent wire length, broke due to the tension, resolder, provide more support on the solder point from circuit board using masking tape

Soldering tip oxidized once dip inside the old flux resin

Day 43 (31/8/2025)

Third day of PID tuning attempts.

Continued fixing wiring.

Day 46 (4/9/2025)

PID tuning still not stable.

Two main problems identified:

Dead zone around $\pm 2^\circ$ (not sensitive).

Robot cannot recover once it falls over, even with high K_i .

Week 8

Day 52 (9/9/2025)

Achieved self-balancing using a verified open-source GitHub codebase.

Previous self-developed code had a hard limitation in structure → no further improvement possible despite tuning

Tips learned:

If tuning no longer improves results, the code structure itself may be the limitation.

Check calibration and reference angles carefully.

Floor alignment affects performance.

Study verified online code structures to see how others process data efficiently.

PID Insights:

Kp → proportional response, sets main reaction strength.

Kd → derivative response, adds damping to counter sudden changes (too much → overshoot).

Ki → integral response, builds correction over time to prevent drift.

For this robot, Kp + Ki are the most critical.

Best approach:

- Find working Kp range.

- Reduce to smallest effective Kp.

- Add Ki to improve recovery.

- Keep Kd small and stable.

Day 54 (11/9/2025)

Designed SolidWorks models for motor, casing, and tire.

Discovered GrabCAD as a valuable resource for ready-made, ISO-standard 3D models.

Learned not all parts need to be modeled from scratch.

Reflection

The project stretched longer than planned due to other commitments and tool/equipment issues. Normally, this could be completed in about **3–4 weeks** of focused work.

Conclusion

This self-balancing robot project has been a rewarding experience in combining electronics, control systems, and mechanical design. Through multiple attempts, I explored both my own PID implementation and an open-source version. While my self-coded approach could balance for only a few seconds, it provided valuable insight into sensor processing and PID tuning. Switching to a verified codebase allowed me to achieve stable balancing and highlighted the importance of efficient code structure in real-time control applications.

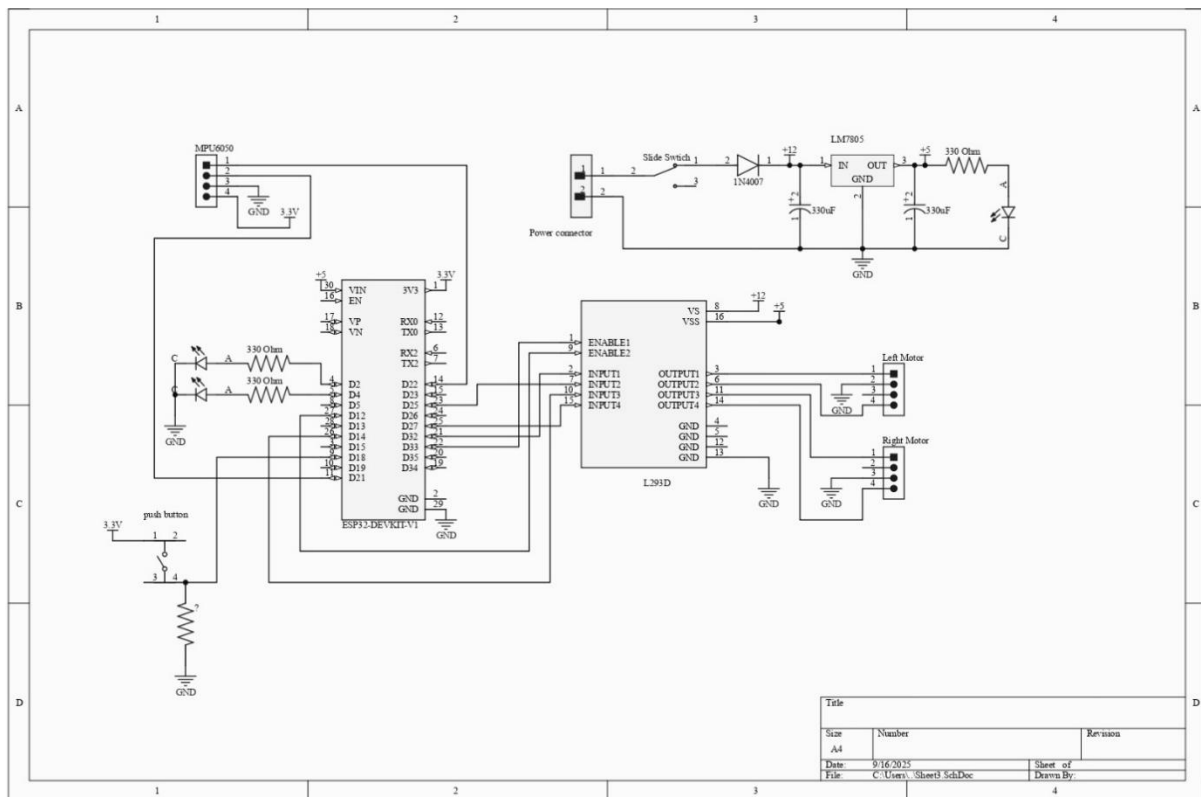
For future improvement, the robot could be upgraded with better sensors like the MPU9250, encoders for precise motor feedback, and a sturdier 3D Printed chassis with optimized weight distribution. On the software side, advanced methods such as adaptive PID or LQR control could further improve stability. Additional features like wireless communication or obstacle detection could also make the robot more interactive and autonomous.

Reference:

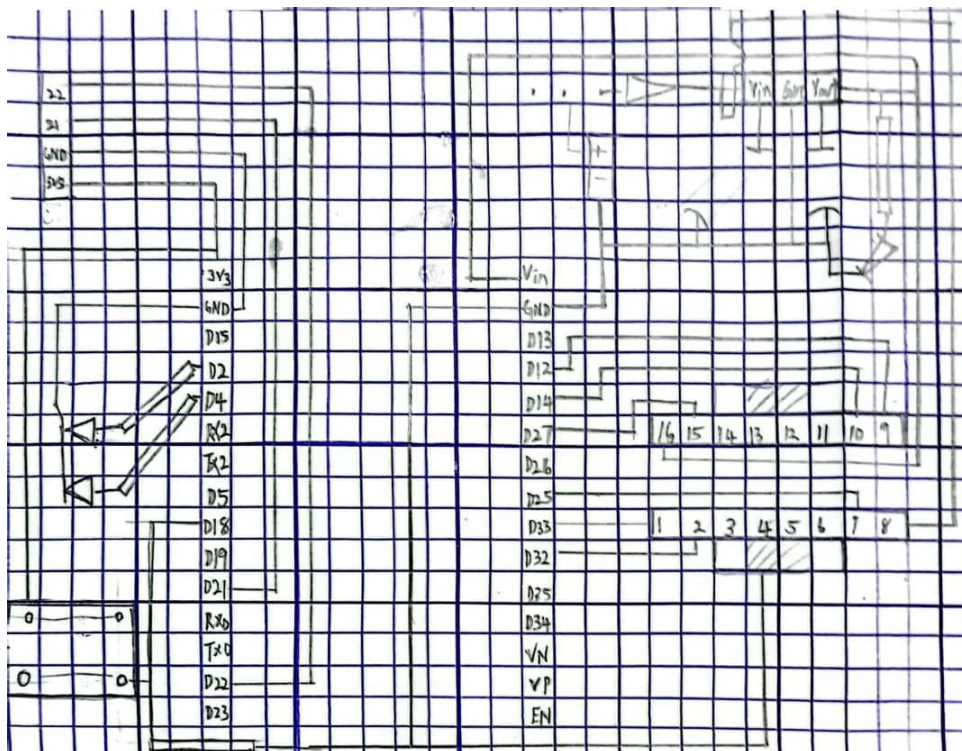
1. <https://www.instructables.com/Inverted-Pendulum-Control-Theory-and-Dynamics/>
2. <https://randomnerdtutorials.com/esp32-pwm-arduino-ide/>
3. <https://youtu.be/pbJbpHjC3v0?si=ynkqsOdQib8Xzfdb>
4. <https://github.com/jrowberg/i2cdevlib>

Appendix

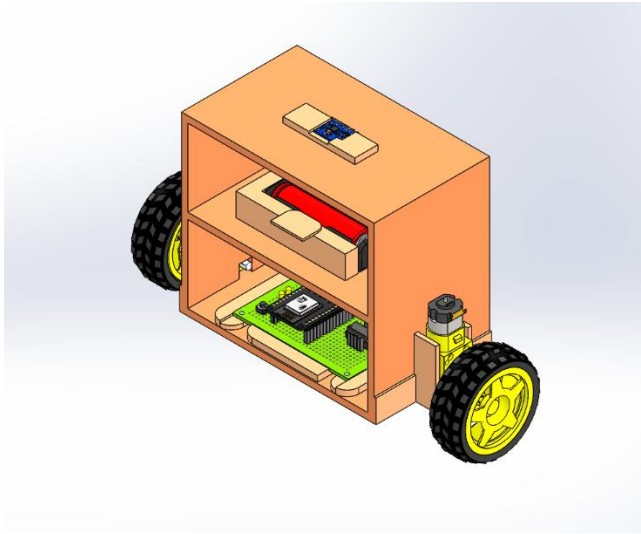
Schematic Diagram



Perfboard layout (30x24)



3D Modeling Solidworks



PID_1.4

```
#include <Adafruit_MPU6050.h>
```

```
#include <Adafruit_Sensor.h>
```

```
#include <Wire.h>
```

```
#include <PID_v1.h>
```

```
Adafruit_MPU6050 mpu;
```

```
// Variables for IMU
```

```
double setpoint= 0; //the value when the bot is perpendicular to ground
```

```
double input, output;
```

```
// PID (Proportional-Derivative for better stability)
```

```
float Kp = 43; // proportional gain 24
```

```
float Kd = 0.44; //derivative gain0.34
```

```
float Ki = 252; // integral gain6.26
```

```
PID pid(&input, &output, &setpoint, Kp, Ki, Kd, DIRECT);
```

```

float pitch = 0, roll = 0;

float delta_time;

unsigned long prev_time = 0;


// Offset (your MPU6050 baseline drift, e.g. 2.2 degrees)
float pitchOffset = 2.9; // you can adjust this value


// Motor pins
const int enablePin = 12; // PWM pin right
const int in1Pin = 14;
const int in2Pin = 27;


const int enablePinB = 33; // PWM pin left
const int in1PinB = 25;
const int in2PinB = 32;


// PWM config
const int freq = 5000;
const int resolution = 8;
int dutyCycle = 190; // 0–255
int pwm;


// ----- IMU UPDATE -----
void updateIMU() {
    sensors_event_t a, g, temp;
    mpu.getEvent(&a, &g, &temp);

```

```

// Calculate delta time

unsigned long current_time = micros(); // CHANGED: micros() for more precise loop timing
delta_time = (current_time - prev_time) / 1e6;
prev_time = current_time;


// Accelerometer angles

float pitch_acc = atan2(a.acceleration.x, sqrt(a.acceleration.y * a.acceleration.y +
                                              a.acceleration.z * a.acceleration.z)) * 180.0 / PI;


// Gyro rates (deg/s)

float gy = g.gyro.y * 180.0 / PI;


// Complementary filter

pitch = 0.98 * (pitch - gy * delta_time) + 0.02 * pitch_acc; // CHANGED: more stable filter
ratio

//pitch = 0.98 * (pitch + gy * delta_time) + 0.02 * pitch_acc;

}


// ----- MOTOR CONTROL -----


void Forward(int pwm) //Code to rotate the wheel forward
{
    digitalWrite(in1Pin, HIGH);
    digitalWrite(in2Pin, LOW);

    digitalWrite(in1PinB, HIGH);

```

```

digitalWrite(in2PinB, LOW);
Serial.print("F"); //Debugging information

ledcWrite(enablePin, pwm);
ledcWrite(enablePinB, pwm);
}
void Reverse(int pwm) //Code to rotate the wheel Backward
{
    digitalWrite(in1Pin, LOW);
    digitalWrite(in2Pin, HIGH);

    digitalWrite(in1PinB, LOW);
    digitalWrite(in2PinB, HIGH);
    Serial.print("R"); //Debugging information

    ledcWrite(enablePin, pwm);
    ledcWrite(enablePinB, pwm);
}
void Stop() //Code to stop both the wheels
{
    digitalWrite(in1Pin, LOW);
    digitalWrite(in2Pin, LOW);

    digitalWrite(in1PinB, LOW);
    digitalWrite(in2PinB, LOW);

    ledcWrite(enablePin, 0);
    ledcWrite(enablePinB, 0);

```

```

    Serial.print("S");

}

// ----- SETUP -----

void setup() {
    Serial.begin(115200);

    pid.SetMode(AUTOMATIC);
    pid.SetSampleTime(10);      // 10ms = 100Hz loop
    pid.SetOutputLimits(-255, 255); // match PWM range

    if (!mpu.begin()) {
        Serial.println("Failed to find MPU6050 chip");
        while (1) delay(10);
    }

    Serial.println("MPU6050 Found!");
    mpu.setAccelerometerRange(MPU6050_RANGE_4_G);
    mpu.setGyroRange(MPU6050_RANGE_500_DEG);
    mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);

    // Motor pin setup
    pinMode(in1Pin, OUTPUT);
    pinMode(in2Pin, OUTPUT);

    ledcAttach(enablePin, freq, resolution);

```

```

pinMode(in1PinB, OUTPUT);
pinMode(in2PinB, OUTPUT);

ledcAttach(enablePinB, freq, resolution);

prev_time = micros(); // CHANGED: start with microseconds
}

// ----- MAIN LOOP -----
void loop() {
    updateIMU();

    input = pitch - pitchOffset;
    pid.Compute();

    // Map PID output (-255 → +255) to PWM magnitude (180 → 255)
    pwm = map(abs(output), 0, 255, 0, 255);

    // Clamp just in case
    pwm = constrain(pwm, 0, 255);

    // if (abs(input)<1.2){
    //   Stop();
    // }

    if (output<0) //Falling towards front
        Forward(pwm); //Rotate the wheels forward
    else if (output>0) //Falling towards back

```



```
Reverse(pwm); //Rotate the wheels backward
```

```
if (input>80 || input<-80){  
  Stop(); //Hold the wheels still  
}
```

```
Serial.print(input);  
Serial.print(",");  
Serial.println(output);  
delay(35);  
}
```

PID_2

```
//Self Balancing Robot

#include <PID_v1.h>

#include <LMotorController.h>

#include "I2Cdev.h"

#include "MPU6050_6Axis_MotionApps20.h"


#define MPU_INT_PIN 23


#if I2CDEV_IMPLEMENTATION == I2CDEV_ARDUINO_WIRE

#include "Wire.h"

#endif


#define MIN_ABS_SPEED 30


MPU6050 mpu;


// MPU control/status vars

bool dmpReady = false; // set true if DMP init was successful

uint8_t mpulntStatus; // holds actual interrupt status byte from MPU

uint8_t devStatus; // return status after each device operation (0 = success, !0 = error)

uint16_t packetSize; // expected DMP packet size (default is 42 bytes)

uint16_t fifoCount; // count of all bytes currently in FIFO

uint8_t fifoBuffer[64]; // FIFO storage buffer


// orientation/motion vars

Quaternion q; // [w, x, y, z] quaternion container

VectorFloat gravity; // [x, y, z] gravity vector
```

```

float ypr[3]; // [yaw, pitch, roll] yaw/pitch/roll container and gravity vector

//PID
double originalSetpoint = 185;
double setpoint = originalSetpoint;
double movingAngleOffset = 0.1;
double input, output;

//adjust these values to fit your own design
double Kp = 30; //55 50 43 30
double Kd = 2.4; //2.6 2.4
double Ki = 142; // 87 97 142
PID pid(&input, &output, &setpoint, Kp, Ki, Kd, DIRECT);

double motorSpeedFactorLeft = 0.9;
double motorSpeedFactorRight = 0.9;

//MOTOR CONTROLLER
int ENA = 12;
int IN1 = 27;
int IN2 = 14;
int IN3 = 32;
int IN4 = 25;
int ENB = 33;

LMotorController motorController(ENA, IN1, IN2, ENB, IN3, IN4, motorSpeedFactorLeft,
motorSpeedFactorRight);

volatile bool mpuInterrupt = false; // indicates whether MPU interrupt pin has gone high

```

```

void dmpDataReady()
{
    mpulInterrupt = true;
}

void setup()
{
    Serial.begin(115200);
    Serial.println("Starting...");
    // join I2C bus (I2Cdev library doesn't do this automatically)
    // #if I2CDEV_IMPLEMENTATION == I2CDEV_ARDUINO_WIRE
    // Wire.begin();
    // TWBR = 24; // 400kHz I2C clock (200kHz if CPU is 8MHz)
    // #elif I2CDEV_IMPLEMENTATION == I2CDEV_BUILTIN_FASTWIRE
    // Fastwire::setup(400, true);
    // #endif
    #if I2CDEV_IMPLEMENTATION == I2CDEV_ARDUINO_WIRE
    Wire.begin(); // use default SDA/SCL pins (ESP32 = GPIO21 SDA, GPIO22 SCL)
    Wire.setClock(400000); // set I2C frequency to 400kHz
    #elif I2CDEV_IMPLEMENTATION == I2CDEV_BUILTIN_FASTWIRE
    Fastwire::setup(400, true);
    #endif

    mpu.initialize();

    devStatus = mpu.dmpInitialize();

```

```

// supply your own gyro offsets here, scaled for min sensitivity
mpu.setXGyroOffset(0);
mpu.setYGyroOffset(0);
mpu.setZGyroOffset(0);
mpu.setZAccelOffset(0); // 1688 factory default for my test chip

// make sure it worked (returns 0 if so)
if (devStatus == 0)
{
// turn on the DMP, now that it's ready
mpu.setDMPEntered(true);

// enable Arduino interrupt detection
attachInterrupt(digitalPinToInterrupt(MPU_INT_PIN), dmpDataReady, RISING);
mpuIntStatus = mpu.getIntStatus();

// set our DMP Ready flag so the main loop() function knows it's okay to use it
dmpReady = true;

// get expected DMP packet size for later comparison
packetSize = mpu.dmpGetFIFOPacketSize();

//setup PID
pid.SetMode(AUTOMATIC);
pid.SetSampleTime(10);
pid.SetOutputLimits(-255, 255);

```

```

Serial.println("DMP ready!");
}
else
{
// ERROR!
// 1 = initial memory load failed
// 2 = DMP configuration updates failed
// (if it's going to break, usually the code will be 1)
    Serial.print("DMP Init failed (code ");
    Serial.print(devStatus);
    Serial.println(")");
}
}

void loop()
{
// if programming failed, don't try to do anything
if (!dmpReady) return;

// wait for MPU interrupt or extra packet(s) available
while (!mpuInterrupt && fifoCount < packetSize)
{
//no mpu data - performing PID calculations and output to motors
pid.Compute();
motorController.move(output, MIN_ABS_SPEED);

}
}

```

```

// reset interrupt flag and get INT_STATUS byte
mpuInterrupt = false;
mpuIntStatus = mpu.getIntStatus();

// get current FIFO count
fifoCount = mpu.getFIFOCount();

// check for overflow (this should never happen unless our code is too inefficient)
if ((mpuIntStatus & 0x10) || fifoCount == 1024)
{
    // reset so we can continue cleanly
    mpu.resetFIFO();
    Serial.println(F("FIFO overflow!"));

// otherwise, check for DMP data ready interrupt (this should happen frequently)
}
else if (mpuIntStatus & 0x02)
{
    // wait for correct available data length, should be a VERY short wait
    while (fifoCount < packetSize) fifoCount = mpu.getFIFOCount();

    // read a packet from FIFO
    mpu.getFIFOBytes(fifoBuffer, packetSize);

    // track FIFO count here in case there is > 1 packet available
    // (this lets us immediately read more without waiting for an interrupt)
    fifoCount -= packetSize;

```

```
mpu.dmpGetQuaternion(&q, fifoBuffer);  
mpu.dmpGetGravity(&gravity, &q);  
mpu.dmpGetYawPitchRoll(ypr, &q, &gravity);  
input = ypr[1] * 180/M_PI + 180;
```

```
Serial.print(input);  
Serial.print(",");  
Serial.println(output);  
}  
}
```